

Transactions, MVCC, and Lock Evidence

Predict state across two sessions, distinguish waiting from work, and communicate an incident.

ACID names transaction properties, not unlimited guarantees

Atomicity

The transaction's defined effects commit together or do not commit.

Boundary matters.

Consistency

A transaction moves between states that satisfy enforced rules when used correctly.

Not automatic business truth.

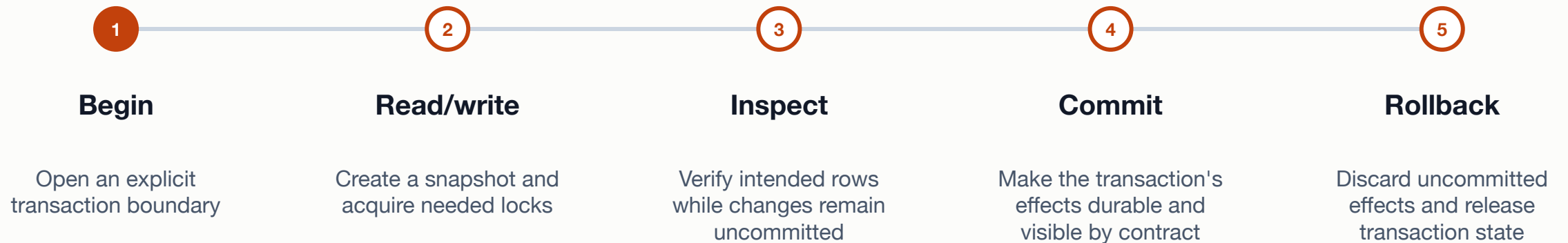
Isolation + durability

Concurrent effects follow an isolation contract; committed effects survive covered failures.

Levels and mechanisms matter.

A transaction moves through observable states

The important boundary is not the query window; it is the transaction state on the server.



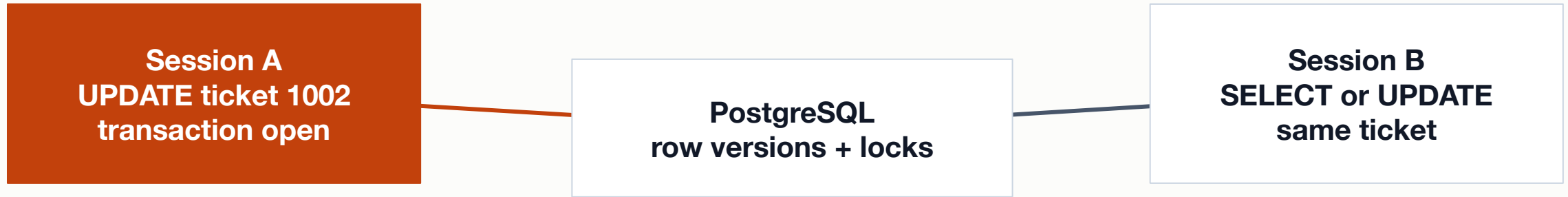
Predict commit and rollback before running the commands

```
BEGIN;  
UPDATE metro_support.tickets  
SET priority = 'urgent'  
WHERE ticket_id = 1002;  
  
SELECT ticket_id, priority  
FROM metro_support.tickets  
WHERE ticket_id = 1002;  
  
-- Choose exactly one:  
COMMIT;  
-- ROLLBACK;
```

Three observations

- What does this session see before completion?
- What can another session see?
- What remains after commit or rollback?

Two sessions can hold different state at the same moment



A read may see a committed version while a conflicting write waits for Session A.

Lab 1: Predict and verify transaction outcomes

Run small commit and rollback cases and account for what each session can observe.

- Predict the value visible inside the transaction and from a second session.
- Run the commit case and verify the final persisted value.
- Run the rollback case and verify that the original value remains.

SUBMIT ONE THING / one transaction-outcome record

A waiting query and an expensive query need different responses

Measure whether work is progressing or waiting before changing the query or index.

MVCC reduces some conflicts; locks still protect coordination

MVCC visibility

- Keeps row versions for concurrent snapshots
- Lets many reads observe committed versions without waiting
- Requires cleanup of obsolete versions

Lock coordination

- Protects conflicting row or object operations
- Can make one transaction wait for another
- Requires ownership and transaction context to diagnose

Slow work consumes resources; blocked work waits

Expensive or slow

- CPU, reads, writes, or network work continues
- Plan and runtime evidence reveal where effort occurs
- Query/model/index changes may reduce work

Blocked or waiting

- Session waits on a conflicting lock or resource
- Wait event and blocker relationship identify ownership
- Resolve transaction state before tuning the query

PostgreSQL can identify waiting and blocking sessions

```
SELECT pid, username, state,  
       wait_event_type, wait_event,  
       xact_start, query_start, query  
FROM pg_stat_activity  
WHERE datname = current_database();
```

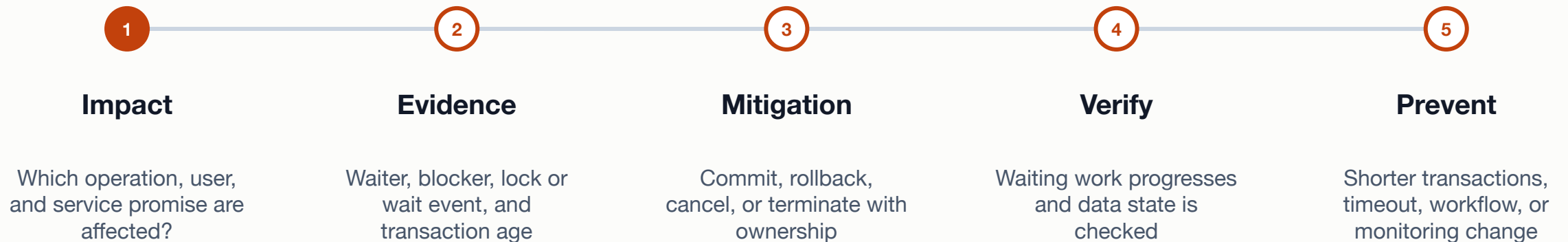
```
SELECT pid, pg_blocking_pids(pid) AS blockers, query  
FROM pg_stat_activity  
WHERE cardinality(pg_blocking_pids(pid)) > 0;
```

Handle with care

- Session metadata may expose sensitive query text.
- PID identifies a connection, not a root cause.
- Confirm ownership before cancellation or termination.

A blocking incident record separates response from diagnosis

Restore service safely, then prevent recurrence with evidence from the transaction lifecycle.



Lab 2: Diagnose a controlled blocking incident

Identify a waiter and blocker, choose a safe mitigation, verify recovery, and write a neutral update.

- Create or inspect the provided two-session blocking state.
- Record waiter, blocker, wait evidence, transaction age, and ownership context.
- Resolve safely, verify progress and data state, then name one prevention step.

SUBMIT ONE THING / one blocking-incident record

Concurrency becomes manageable when state and ownership are visible

Predict transaction state, distinguish work from waiting, and verify the system after intervention.

- Which transaction owns the conflicting state?
- What does the waiting session need?
- What proves the mitigation restored safe progress?